PHANTOM · A FIELD GUIDE

The 10 codebases every engineer should be able to read.

Why they're worth studying, what you'll learn, and where to start.

10 repos · ~25 min read

From the team building Phantom.
phantom.video

INTRO

Why these ten.

There's no shortage of 'best codebases' lists. Most are about the popular projects. This is a list about the ones worth reading — the ones whose internals teach you something general about how systems are built.

The repos picked here are all open. They're all readable in a weekend (or, in Postgres's case, a month of weekends — but the first week pays for itself). And each one demonstrates a pattern that shows up everywhere once you've seen it.

For each repo, you'll get: why it's worth your time, the dominant architectural pattern, three or four concrete things you'll learn, and the single file to open first.

If you want any of these as a 3-minute video walkthrough, generate one at phantom.video. The free tier gets you the first video.

React

The UI library everyone copies the patterns from.

WHY IT'S WORTH YOUR TIME

React's reconciler is one of the cleanest decoupled architectures in frontend. Reading it teaches you scheduling, double-buffered state, and the cost of immutability done well.

DOMINANT PATTERN

Fiber tree + reconciler + renderer separation

WHAT YOU'LL LEARN

- Why hooks are stored as a linked list on the fiber
- How the work loop yields to the browser between units of work
- The contract between reconciler and renderer (react-dom, react-native)
- How Suspense leverages throw-based control flow

START HERE

`packages/react-reconciler/src/ReactFiberWorkLoop.js`

PostgreSQL

The database that ate the database world.

WHY IT'S WORTH YOUR TIME

Postgres is one of the most thoroughly engineered codebases in existence. Forty years of careful C, and it's still readable. The query planner alone is worth a week.

DOMINANT PATTERN

Process-per-connection, shared memory + WAL, extensible types

WHAT YOU'LL LEARN

- How the query planner enumerates and costs plan trees
- Why MVCC (multi-version concurrency control) avoids most locks
- How extensions can register new types without touching core
- The WAL contract that lets replication work

START HERE

`src/backend/executor/execMain.c`

Redis

In-memory data structures at network speed.

WHY IT'S WORTH YOUR TIME

Redis is the cleanest example of 'do one thing well' in systems code. The event loop is 1,200 lines. Every line earns its place.

DOMINANT PATTERN

Single-threaded event loop + on-disk persistence options

WHAT YOU'LL LEARN

- Why single-threaded is fast (and where it isn't)
- How RDB snapshots and AOF logging trade off durability vs. speed
- Sorted set implementation: skip list + hash table dual indexing
- Cluster mode's gossip protocol and slot remapping

START HERE

`src/server.c`

Vue 3

The framework where the reactivity is the point.

WHY IT'S WORTH YOUR TIME

Vue 3 was a ground-up rewrite around ES Proxies. Reading it is a masterclass in reactive systems — useful even if you never write Vue.

DOMINANT PATTERN

Proxy-based reactivity + compiler-emitted hyperscript

WHAT YOU'LL LEARN

- How Proxies enable fine-grained dependency tracking
- Why the Composition API is 'just' function composition over refs
- Compiler optimizations: static hoisting, patch flags
- The reactivity scheduler and effect dedup

START HERE

`packages/reactivity/src/reactive.ts`

Bun

A JS runtime written in Zig that's faster than Node.

WHY IT'S WORTH YOUR TIME

Bun is the most interesting language-runtime project of the last five years. Reading it teaches you Zig, FFI design, and how to build Node-compatibility without becoming Node.

DOMINANT PATTERN

JavaScriptCore (not V8) + native HTTP server in Zig

WHAT YOU'LL LEARN

- Why Bun chose JavaScriptCore over V8
- How Bun.serve is a Zig HTTP server with a JS surface
- Node compatibility as a deliberate, growing surface
- The bundler architecture (Esbuild-inspired but native)

START HERE

`src/bun.js/javascript.zig`

FastAPI

What Flask should have been if it grew up after type hints.

WHY IT'S WORTH YOUR TIME

FastAPI is a thin layer over Starlette. Reading it shows how much leverage you get from Pydantic + type hints — the dependency tree is resolved at request time without metaclass voodoo.

DOMINANT PATTERN

Type-driven routing + dependency injection over Starlette

WHAT YOU'LL LEARN

- How type hints drive validation, docs, and DI
- The request lifecycle: dependency tree resolution
- Why FastAPI generates OpenAPI 'for free'
- Where Starlette's async primitives end and FastAPI begins

START HERE

`fastapi/routing.py`

Tailwind CSS

A class-name system that turned out to be a design system.

WHY IT'S WORTH YOUR TIME

The JIT engine is the second-best PostCSS plugin ever written. Reading it teaches you AST manipulation, source-driven generation, and how to build configurable design systems that don't drown.

DOMINANT PATTERN

PostCSS plugin + JIT scan + theme-driven utility generation

WHAT YOU'LL LEARN

- How JIT scans source files and emits only used utilities
- Why the theme is the source of truth, not the CSS
- The variant system: how `hover:` and `dark:` compose
- The new Oxide engine: why rewrite the core in Rust

START HERE

`src/lib/setupTrackingContext.js`

Next.js

The framework that ate the meta-framework world.

WHY IT'S WORTH YOUR TIME

Next.js is enormous, but the App Router subsystem is the most interesting bit of frontend architecture this decade. Read it for how RSCs actually serialize between server and client.

DOMINANT PATTERN

App Router + RSCs + Turbopack/Webpack

WHAT YOU'LL LEARN

- How RSCs render on the server and hydrate on the client
- Why layouts, loading, and error boundaries unify under one tree
- Turbopack's incremental compilation model
- How edge runtime differs from Node runtime (and why both)

START HERE

`packages/next/src/server/app-render/app-render.tsx`

LangChain

The 'glue code' library that became its own glue.

WHY IT'S WORTH YOUR TIME

Whatever you think of LangChain, the LCEL (LangChain Expression Language) abstraction is genuinely interesting. Reading it shows you how to compose async streams of typed values cleanly.

DOMINANT PATTERN

Runnable interface + LCEL pipe composition

WHAT YOU'LL LEARN

- Why every primitive is a Runnable
- How the pipe operator composes runnables with type safety
- Streaming: token-by-token output through the chain
- Tool calling: unifying agent and chain APIs

START HERE

`libs/core/langchain_core/runnables/base.py`

10 / 10

Sourcegraph

Code search at the scale of all of GitHub.

WHY IT'S WORTH YOUR TIME

Sourcegraph is the only OSS project that's seriously attempted universal code search. Reading the indexer + query planner is a masterclass in trade-offs at scale.

DOMINANT PATTERN

Distributed indexer (Zoekt) + GraphQL API + frontend

WHAT YOU'LL LEARN

- How Zoekt indexes billions of lines with sub-second search
- The trade-off between exact vs. regex vs. structural search
- Why GraphQL fits a code-intelligence API surface
- How to expose code intelligence (definitions, references) at scale

START HERE

`cmd/frontend/main.go`

ONE MORE THING

Want a video of any of these?

Generate a 3-minute narrated walkthrough of any repo on this list (or your own codebase) at phantom.video. First one's free.

phantom.video →

No signup required for your first video.